

Data Handling in R

2026-09-07

Abstract Corrections of and comments on the syllabus text, the example scripts, and answers to selected assignments are solicited. Requests for additions are welcome as well. Future students will appreciate your input! We acknowledge valuable contributions by former teachers Ozan Aksoy, Jos Dessens, Wim Jansen, Peter van de Heijden, Peter Ligtig, David Macro, Vardan Barsegyan, and Weverthon Machado as well as by numerous students.

Table of contents

1	Introduction	2
2	Getting to know and summarizing data	2
2.1	R	3
2.2	Assignments	9
3	Documenting data	10
3.1	R	10
3.2	Assignments	11
3.3	Answers	12
3.3.a	1. Rename the variable <code>hrespnr</code> to <code>hid</code> and the variable <code>hrinc</code> to <code>hinc</code> in one command.	12
3.3.b	2. The variable <code>sex</code> is coded numerically. Convert <code>sex</code> into a factor with meaningful labels. Use <code>male</code> and <code>female</code> as category labels.	12
3.3.c	3. The variables <code>sexrol1</code> and <code>sexrol2</code> measure family attitudes. Convert both variables into factors with the following category labels: fully disagree, slightly disagree, indifferent, slightly agree, fully agree.	12
4	Creating new variables	13
4.1	R	13
4.2	Assignments	15
4.2.a	Create dummy variables.	15
4.2.b	Recoding a categorical variable.	16
4.2.c	Recode a continuous variable into groups.	16
4.2.d	Centering and standardizing variables.	17
4.2.e	Creating additive scales.	17
4.3	Answers	18

4.3.a Create dummy variables	18
4.3.b Recoding a categorical variable	20
4.3.c Recode a continuous variable into groups	21
4.3.d Centering and standardizing variables	22
4.3.e Creating additive scales	25
5 Selecting cases and repeating analyses for groups	29
5.1 R	29
5.2 Assignments	31
5.3 Answers	32
5.3.a 1. Counting observations with <code>filter()</code> and <code>nrow()</code>	33
5.3.b 2. Working with a subset	35
5.3.c 3. A selection variable	36
5.3.d 4. Comparing three commands	38
5.3.e 5. Working within households	39

1 Introduction

- Contents via shortcode (see below)

2 Getting to know and summarizing data

In this part you will learn about the following basic tasks:

- Change your working directory/folder.
- Load a dataset from disk into computer memory.
- Explore the variables in the data.
- List values for (some) variables.
- Sort the observations on variables.
- The concept of “key variable(s)”.
- Create frequency distributions of variables.
- Create cross tabulations of multiple variables.
- Produce a table of summary statistics (e.g., means, sd = standard deviation) for a series of variables.
- Produce a table of correlations of quantitative variables.

The concept of key variables may be new to you. A key consists of one or more variables that uniquely identify observations.

In social science datasets, we often use single observations to represent all variables measured on subjects. Each person is then a row in the data matrix. Such datasets often contain a respondent number, say `id`, usually integer valued. The values of `id` are usually unique, and then `id` qualifies

as a key. If you know a subject's value of `id`, you can find the corresponding row in the data matrix, and hence know all the subject's characteristics.

If a value of `id` occurs more than once, obviously, the value of `id` does not identify a unique subject, and `id` is not a key. Observe that `id` need not take all possible values. For instance, if your `id` consists of 9 digits, likely your dataset does not contain all possible numbers of 9 digits. Many possible values of `id` will not be used. That is not a problem. `id` would still be a key if all values are distinct.

If `id` takes duplicate values, we may form a key by considering additional variables. For instance, think of a household survey with one or two adults. Observations are people nested in households. The variable `hid`, household number, would not be a key. The combination of `hid` and `sex` might be a key if the sample comprises single people and different-sex couples, but not if the sample also includes same-sex couples, cohabiting siblings, etc. In the latter case, the data may contain a variable indicating whether a person is the oldest adult in the household, and together with `hid` this variable may form a key. If you only have birth dates, the combination of `hid` and birth date is usually a key, but you could have bad luck if your data include a household with two cohabiting twin sisters.

If a dataset does not contain a case identifier such as a respondent number, we strongly suggest you create one. Be sure to do this in a reproducible way. First put the observations in a specific order, by sorting on a series of variables that uniquely identify cases. In the example below, we assume that each observation has a unique pattern on the variables `city`, `age`, `edu`, and `sex`. Obviously, you should verify this assumption.

In R, you can check duplicates and create an identifier as follows:

```
df |>
  count(city, age, edu, sex) |>
  filter(n > 1)

df <- df |>
  arrange(city, age, edu, sex) |>
  mutate(respnr = row_number())
```

After sorting the cases, a new variable is created with the value 1 in the first observation, 2 in the second observation, 3 in the third observation, etc. This is one of the occasions in which you may want to save the dataset, but we urge you not to overwrite the original file.

2.1 R

You will learn the R functions listed below to describe and summarize data. Read the corresponding documentation pages when needed. Start with the description and examples. We do not expect you to learn all functions by heart, but you should get a rough idea of, and remember, what they can accomplish.

Function	Description
<code>read_csv()</code>	Load a CSV file into R
<code>read_dta()</code> / <code>read_sav()</code>	Load a Stata / SPSS file into R
<code>read_excel()</code>	Load an Excel file into R
<code>glimpse()</code>	Describe data contents
<code>skim()</code>	Describe data contents and summary statistics
<code>View()</code>	View observations and variables
<code>arrange()</code>	Sort observations
<code>table()</code>	One-way and two-way tables of frequencies
<code>prop.table()</code>	Convert frequencies to proportions
<code>tabyl()</code>	Frequency tables with percentages
<code>chisq.test()</code>	Pearson's chi-square test for independence
<code>summarise()</code>	Summary statistics
<code>cor()</code>	correlations (Pearson by default)
<code>count()</code>	Count observations by category or group
<code>filter()</code>	Select observations satisfying a condition

Examples:

Load a file into memory, using a relative path:

```
library(readr) # for opening CSV files
data <- read_csv("data/mydata.csv")

library(haven) # for opening Stata (.dta) and SPSS (.sav) files
data <- read_dta("data/mydata.dta")
data <- read_sav("data/mydata.sav")

library(readxl) # for opening Excel (.xlsx) files
data <- read_excel("data/mydata.xlsx")
```

Describe variables in the active dataset:

```
glimpse(data) # shows basic info about the dataset and
              # a overview of all variables

data |>
  select(edu, exp) |>
```

```

glimpse() # compact overview of edu and exp

data |>
  select(starts_with("rat")) |>
  glimpse() # describe all variables starting with rat

skim(data) # summary statistics (from package 'skimr')
View(data) # open data viewer in RStudio

```

List observations:

```

data # all variables and all observations

data |>
  select(edu, inc) # selection of variables

data |>
  filter(country == 4) # selection of observations
                        # (all rows in which variable 'country' equals 4)

data |>
  select(country, edu, inc) |> # selection of variables
  slice(1:20)                 # selection of observations 1 to 20

```

Sort observations on one or more variables:

```

data |>
  arrange(edu) # sort by edu (increasing)

data |>
  arrange(edu, inc) # sort by edu and inc (increasing)

data |>
  arrange(desc(edu)) # sort by edu (decreasing)

```

One-way tabulation, frequencies:

```

table(data$class) # one-way table

table(data$edu, useNA = "ifany") # one-way table, include missing values

# usign tabyl, from package 'janitor'

```

```
data |>
  tabyl(edu) # frequency table with missing values and percentages
```

Two-way tabulation of variables:

```
table(data$edu, data$class)

# using tabyl:
data |>
  tabyl(edu, class)
```

Row and column percentages:

```
# row percentages:
table(data$edu, data$class) |>
  prop.table(margin = 1)
# column percentages:
table(data$edu, data$class) |>
  prop.table(margin = 2)

# using tabyl:
data |>
  tabyl(edu, class) |>
  adorn_percentages("row")

data |>
  tabyl(edu, class) |>
  adorn_percentages("columns") |>
  adorn_pct_formatting() # optional
```

In `prop.table`, the argument `margin = 1` computes proportions within rows, while `margin = 2` computes proportions within columns.

Pearson's chi-square test for independence:

```
table(data$edu, data$class) |>
  chisq.test()
# Pearson's chi-square test for independence
```

The chi-square test checks whether two categorical variables are statistically independent.

Tabulation of three or more variables:

```

# three variables
xtabs(~ edu + class + gender, data = data, addNA=TRUE)

data |>
  tabyl(edu, class, gender)

# four variables (not possible with tabyl)
xtabs(~ edu + class + gender + country, data = data, addNA=TRUE)

# It is also possible to obtain frequencies using count().
# But it is not printed as a table and it does not show missing combinations
data |>
  count(gender, edu, class)

data |>
  count(country, gender, edu, class)

```

Summary statistics:

```

# Summary statistics for income
data |>
  skim(income)

data |>
  summarise(
    mean_income = mean(income, na.rm = TRUE),
    sd_income = sd(income, na.rm = TRUE),
    min_income = min(income, na.rm = TRUE),
    max_income = max(income, na.rm = TRUE)
  )

```

Correlation matrix of two or more variables:

```

data |>
  select(edu, inc, age) |>
  cor(use = "complete.obs") # Pearson correlations

```

Count the number of observations meeting a condition:

```

# count observations with income above 50000:
data |>
  filter(income > 50000) |>
  nrow()

```

```

# count observations with income between 5000 and 50000, boundaries excluded:
data |>
  filter(income > 5000, income < 50000) |>
  nrow()

# count observations with income between 5000 and 50000, boundaries included:
data |>
  filter(income >= 5000, income <= 50000) |> # alternative: filter(between(income,
5000, 50000))
  nrow()

# count observations with income above 50000 and income not missing:
data |>
  filter(income > 50000, !is.na(income)) |>
  nrow()

# count observations with income above 50000 and sex equal to 1
data |>
  filter(income > 50000, sex == 1) |>
  nrow()

# count observations with income above 50000 OR sex equal to 1
data |>
  filter(income > 50000 | sex == 1) |>
  nrow()

```

Check that the variable `birthd` is a key:

```

data |>
  summarise(no_missing = all(!is.na(birthd))) # birthd is never missing

data |>
  count(birthd) |>
  filter(n > 1) # check duplicates in birthd

```

Check that `birthd` and `sex` form a composite key:

```

data |>
  summarise(no_missing = all(!is.na(birthd), !is.na(sex))) # birthd and sex are
never missing

data |>

```



```
count(birthd, sex) |>
filter(n > 1) # check duplicates in the combination of birthd and sex
```

Create a key:

```
data <- data |>
  arrange(birthd, sex) |>
  mutate(id = row_number()) # create id after sorting by birthd and sex
```

2.2 Assignments

In these assignments we use the dataset `sbs399.dta`. Solutions can be found in the file `dh_summary_assignments.Rmd`. Be sure to start your R script in the style of the template files.

1. How many observations and variables the dataset contains?
2. What is the variable label of `feduc`?
3. Display a frequency distribution of `educ`. What is the percentage of subjects with an education up to category 4? How many missing values are there in `educ`?
4. Select the variables `hrinc` and `pres`. Adjust the syntax to show these variables only for men among the initial 40 observations. Further adjust the syntax to restrict the sample to men with below median education (see previous question) among the initial 40 observations. Finally, show the variables `hrinc` and `pres` for men and women among the initial 5 observations and arrange the results by sex.
5. List values of `age` in increasing order. Next, list just the five smallest values, and list just the five largest values. Do not read these values from the data viewer. Give syntax to display the requested cases.
6. Create a two-way table of `feduc` rows and `meduc` columns. Add row and column percentages. Test for independence of `feduc` and `meduc` using Pearson's chi-square test.
7. What are the mean, standard deviation, median, minimum, and maximum value of `feduc`?
8. Verify that the number of subjects for whom the mother is higher educated than the father is 543. What is the count if you restrict to female subjects?
9. What are the Pearson correlations of `educ`, `feduc`, and `meduc`? Use the help system to find out how to compute Spearman's rank correlations.
10. Is it true that for any variable the values are above average for 50% of cases? If you are uncertain, check this with some variables. For an explanation, think about this example: what percentage of people have more than the average number of legs? 50%? Who are these people?
11. Is `hrespnr` a key, meaning that it is never missing and all values are unique? Why? Do not rely on the data viewer. Write syntax to obtain the information on which you base your answer.

What about the pair of variables `hrespnr` and `sex`? Why? What about `hrespnr`, `sex`, and `age`, and `hrespnr`, `sex`, and `fage`?

3 Documenting data

In this short section you will learn about the following basic tasks:

- Rename variables and give them clear names.
- Use meaningful names for categorical values.
- Convert categorical variables to factors.
- Code missing values as `NA`.
- Change the order of variables in a dataset.

3.1 R

Learn the following R commands to document the data, variables, and values:

Function	Description
<code>rename()</code>	Rename variables
<code>mutate()</code>	Modify or create variables
<code>factor()</code>	Convert a variable to a factor
<code>na_if()</code>	Change a specific value to <code>NA</code>
<code>replace_na()</code>	Replace <code>NA</code> with a specific value
<code>relocate()</code>	Change the order of variables

Rename variables:

```
data <- data |>
  rename(educ = v101)
# rename v101 to educ

data <- data |>
  rename(sex = v1, edu = v2)
# rename multiple variables
```

Convert a categorical variable to a factor:

```
data <- data |>
  mutate(
    sex = factor(
      sex,
      levels = c(1, 2),
```

```

    labels = c("male", "female")
  )
)
# assign meaningful labels to categories

```

Code missing values:

```

data <- data |>
  mutate(educ = na_if(educ, 99))
# change the value 99 to NA

data <- data |> mutate(across(c(educ, feduc), ~ na_if(.x, 9)))
# change the value 9 to NA in educ and feduc

```

Replace missing values with a numeric code:

```

data <- data |>
  mutate(educ = replace_na(educ, 99))
# change NA to 99

```

Change the order of variables:

```

data <- data |>
  relocate(hrespnr, sex, age, edu)
# put these variables first

```

Be careful when replacing missing values with numeric codes such as 99. In R, missing values should usually be coded as `NA`. Numeric missing-value codes can easily be treated as real values in analyses if they are not recoded properly.

3.2 Assignments

See `dh_doc.Rmd` for solutions.

In these assignments we use the dataset `sbs399_unlabeled.dta`.

1. Rename the variable `hrespnr` to `hid` and the variable `hrinc` to `hinc` in one command.
2. The variable `sex` is coded numerically. Convert `sex` into a factor with meaningful labels. Use `male` and `female` as category labels.
3. The variables `sexrol1` and `sexrol2` measure family attitudes. Convert both variables into factors with the following category labels:
 1. fully disagree
 2. slightly disagree

3. indifferent
4. slightly agree
5. fully agree

3.3 Answers

3.3.a 1. Rename the variable `hrespnr` to `hid` and the variable `hrinc` to `hinc` in one command.

```
data <- data |>
  rename(
    hid = hrespnr,
    hinc = hrinc
  )
```

3.3.b 2. The variable `sex` is coded numerically. Convert `sex` into a factor with meaningful labels. Use `male` and `female` as category labels.

```
data <- data |>
  mutate(
    sex = factor(
      sex,
      levels = c(1, 2),
      labels = c("male", "female")
    )
  )
```

3.3.c 3. The variables `sexrol1` and `sexrol2` measure family attitudes. Convert both variables into factors with the following category labels: fully disagree, slightly disagree, indifferent, slightly agree, fully agree.

```
data <- data |>
  mutate(
    sexrol1 = factor(
      sexrol1,
      levels = c(1, 2, 3, 4, 5),
      labels = c(
        "fully disagree",
        "slightly disagree",
        "indifferent",
        "slightly agree",
        "fully agree"
      )
    ),
```

```
sexrol2 = factor(
  sexrol2,
  levels = c(1, 2, 3, 4, 5),
  labels = c(
    "fully disagree",
    "slightly disagree",
    "indifferent",
    "slightly agree",
    "fully agree"
  )
)
```

4 Creating new variables

In this part you will learn about the following basic tasks:

- Creating a dummy (0/1) variable.
- Recoding a continuous variable.
- Recoding a continuous variable into groups.
- Centering and standardizing variables.
- Reverse coding of variables.
- Creating additive scales.

4.1 R

Learn the following R commands to create new or modify existing variables.

table

Examples:

Create a variable as a mathematical expression of existing variables:

```
data <- data |>
  mutate(
    hhinc = h_inc + w_inc,
    minc = (h_inc + w_inc) / 2,
    adiff = abs(age - page),
    inc2 = inc^2
  )
# sum, mean, absolute difference, and squared income
```

Change negative values of an existing variable:

```
data <- data |>
  mutate(
    inc = if_else(inc < 0, 0, inc)
  )
# replace negative values by 0
```

Create a dummy variable:

```
data <- data |>
  mutate(
    male = if_else(sex == 2, 1, 0)
  )
# create dummy variable male
```

Preserve missing values:

```
data <- data |>
  mutate(
    male = if_else(
      is.na(sex),
      NA_real_,
      if_else(sex == 2, 1, 0)
    )
  )
# sex = NA implies male = NA
```

Recode a dummy variable:

```
data <- data |>
  mutate(
    female = if_else(is.na(male), NA_real_, 1 - male)
  )
# male = 1 becomes female = 0 and vice versa
```

Recode values into categories:

```
data <- data |>
  mutate(
    region = case_when(
      country %in% c(71, 72, 79) ~ 1,
      between(country, 73, 78) ~ 2,
      !is.na(country) ~ 3,
      TRUE ~ NA_real_
    )
  )
```

```

    )
  )
  # create region variable

```

Count the number of variables equal to a specific value:

```

data <- data |>
  mutate(
    ngood = rowSums(
      across(good01:good05, ~ .x == 2),
      na.rm = TRUE
    )
  )
# count how many variables equal 2

```

Create a standardized variable:

```

data <- data |>
  mutate(
    zedu = (edu - mean(edu, na.rm = TRUE)) /
      sd(edu, na.rm = TRUE)
  )
# standardize edu

```

Create a rank variable:

```

data <- data |>
  mutate(
    rinc = rank(inc)
  )
# rank order statistics of income

```

4.2 Assignments

4.2.a Create dummy variables.

In the dataset `sbs399` the variable `class` marks the respondent's social class. How often is this variable non-missing?

We want to create a dummy variable for being of white collar. This variable, which we will call `white`, is 1 for subjects belonging to the white-collar class and 0 otherwise. We interpret missing values in `class` to denote “unknown”. Thus, the dummy `white` should also be missing if `class` is missing.

- Create this dummy variable using `if_else()`. Make sure that missing values are handled correctly.
- Create the same dummy variable using `case_when()`.
- Produce a cross-tabulation of the two newly created variables and verify that they are identical.

4.2.b Recoding a categorical variable.

In the dataset `sbs399` the variable `polit` marks the political party the respondent intended to vote for in a national election.

(a) Generate a new variable `P` in which political preference is expressed on a left-to-right scale:

- 1 = SP
- 2 = GroenLinks
- 3 = PvdA
- 4 = D66
- 5 = CDA
- 6 = VVD
- 7 = GPV
- 8 = RPF
- 9 = SGP
- 10 = Unie 55+
- 11 = AOV
- 12 = CD

Use `case_when()` to construct the variable.

(b) Recode `P` into a new variable `P3` in which political preference is expressed in three categories:

- 1 = Left (SP, GroenLinks, PvdA, D66)
- 2 = Center (CDA)
- 3 = Right (all remaining parties)

4.2.c Recode a continuous variable into groups.

We consider the hourly wage variable `hrinc`.

Generate a new variable `Q` for the four quartiles of `hrinc`:

- `Q = 1` for the lowest 25%
- `Q = 2` for the next 25%
- `Q = 3` for the next 25%
- `Q = 4` for the highest 25%

Verify that the groups contain approximately equal numbers of observations.

4.2.d Centering and standardizing variables.

Use the dataset `sbs399`. We focus on `hrinc`.

- (a) Create a variable `c_hrinc` that contains the centered hourly wage rate by subtracting the sample mean of `hrinc`.

Check that:

- the mean of `c_hrinc` is approximately 0;
 - the standard deviations of `c_hrinc` and `hrinc` are the same.
- (b) Create a variable `z_hrinc` that contains the standardized hourly wage rate by subtracting the mean of `hrinc` and dividing by its standard deviation.

Check that:

- the mean of `z_hrinc` is approximately 0;
 - the standard deviation of `z_hrinc` is approximately 1.
- (c) Create a variable `z2_hrinc` that standardizes hourly income separately for men and women.

Ensure that:

- you obtain one variable, not two;
 - the variable is not missing unless `hrinc` itself is missing.
- (d) Apply a unit-range transformation to `hrinc`:

$$U(x) = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Verify that the transformed minimum equals 0 and the transformed maximum equals 1.

Which method do you prefer for income? And for number of hours worked per week?

4.2.e Creating additive scales.

We consider the five items `sexrol1` to `sexrol5` in the dataset `sbs399`.

- (a) Explain why it is not sensible to simply add the five items.
- (b) One of the items should be reverse coded. Which one?

Construct a new variable for this reversed item using:

- a recoding approach with `case_when()`;
- a mathematical expression.

Verify that both constructed variables are identical.

- (c) Create the attitude measure from the five items.

Hint: use `rowMeans()`.

What is the mean of the measure? Can you interpret this mean substantively?

- (d) The data used so far are complete, meaning that respondents answered all items. In many surveys, however, respondents do not answer every question. This situation is illustrated by the dataset `sbs399_sexroles`.

We will now discuss a simple ad hoc method for dealing with missing values when constructing a scale. Although more advanced methods are available and will be discussed later in the course, this approach remains useful for understanding the basic idea.

We simply take the mean of all valid item responses for each respondent. Thus, if a respondent answered items 1, 2, and 3, but not items 4 and 5, we calculate the scale score as the mean of items 1, 2, and 3. If a respondent answered all items, we take the mean across all items. If a respondent did not answer any of the items, the scale score should be missing as well.

In R, this can be accomplished using `rowMeans()` together with the argument `na.rm = TRUE`, which computes the mean using all available values while ignoring missing values.

4.3 Answers

Make sure this file is in the same R Project as the `data` folder. Use relative file paths, not full paths to your computer.

```
library(tidyverse)
library(haven)

sbs399 <- read_dta("data/sbs399.dta")
```

4.3.a Create dummy variables

How often is `class` non-missing?

```
sbs399 |>
  filter(!is.na(class)) |>
  nrow()
```

```
[1] 1969
```

```
# number of non-missing values in class
```

Check the categories of `class` before creating the dummy variable.

```
table(sbs399$class, useNA = "ifany")
```

1	2	3	<NA>
1447	494	28	1381

```
# frequency table of class, including missing values
```

Create the dummy variable `white` using `if_else()`.

```
sbs399 <- sbs399 |>
  mutate(
    white = if_else(
      is.na(class),
      NA_real_,
      if_else(class == 2, 1, 0)
    )
  )
# white = 1 for white-collar class, white = 0 otherwise, class = NA implies white
# = NA
```

Create the same dummy variable using `case_when()`.

```
sbs399 <- sbs399 |>
  mutate(
    white2 = case_when(
      class == 2 ~ 1,
      !is.na(class) ~ 0,
      TRUE ~ NA_real_
    )
  )
# same dummy variable, created with case_when()
# change class == 2 if your frequency table shows a different code for white collar
```

Produce a cross-tabulation of the two newly created variables.

```
table(sbs399$white, sbs399$white2, useNA = "ifany")
```

	0	1	<NA>
0	1475	0	0
1	0	494	0
<NA>	0	0	1381

```
# the two variables should be identical
```

4.3.b Recoding a categorical variable

Check the original party codes before recoding.

```
table(sbs399$polit, useNA = "ifany")
```

```
 1  2  3  4  5  6  7  8  9 10 11 12 13  
622 524 632 929 199  54  63  30  17  23  45  25 187
```

```
# frequency table of polit; verify which code belongs to which party
```

Generate a new variable P in which political preference is expressed on a left-to-right scale.

```
sbs399 <- sbs399 |>  
  mutate(  
    P = case_when(  
      polit == 1 ~ 1, # SP  
      polit == 2 ~ 2, # GroenLinks  
      polit == 3 ~ 3, # PvdA  
      polit == 4 ~ 4, # D66  
      polit == 5 ~ 5, # CDA  
      polit == 6 ~ 6, # VVD  
      polit == 7 ~ 7, # GPV  
      polit == 8 ~ 8, # RPF  
      polit == 9 ~ 9, # SGP  
      polit == 10 ~ 10, # Unie 55+  
      polit == 11 ~ 11, # AOV  
      polit == 12 ~ 12, # CD  
      TRUE ~ NA_real_  
    )  
  )  
# create left-to-right political preference scale  
# here the original codes already run from left to right, so P equals polit  
# adjust the mapping if your frequency table shows a different ordering
```

Check the new variable.

```
table(sbs399$P, useNA = "ifany")
```

1	2	3	4	5	6	7	8	9	10	11	12	<NA>
622	524	632	929	199	54	63	30	17	23	45	25	187

```
# frequency table of P
```

Recode P into a new variable P3 with three categories.

```
sbs399 <- sbs399 |>
  mutate(
    P3 = case_when(
      P %in% c(1, 2, 3, 4) ~ 1,
      P == 5 ~ 2,
      !is.na(P) ~ 3,
      TRUE ~ NA_real_
    )
  )
# P3 = 1 Left, P3 = 2 Center, P3 = 3 Right
```

Check the new variable.

```
table(sbs399$P3, useNA = "ifany")
```

1	2	3	<NA>
2707	199	257	187

```
# frequency table of P3
```

4.3.c Recode a continuous variable into groups

Generate a new variable Q for the four quartiles of hrinc.

```
sbs399 <- sbs399 |>
  mutate(
    hrinc_rank = rank(hrinc, na.last = "keep") / sum(!is.na(hrinc)),
    Q = case_when(
      hrinc_rank <= .25 ~ 1,
      hrinc_rank <= .50 ~ 2,
      hrinc_rank <= .75 ~ 3,
      !is.na(hrinc_rank) ~ 4,
      TRUE ~ NA_real_
    )
  )
```

```

    )
  )
  # create four approximately equal groups of hrinc
  # na.last = "keep" makes sure missing values on hrinc stay missing on Q

```

Verify that the groups contain approximately equal numbers of observations.

```
table(sbs399$Q, useNA = "ifany")
```

```

  1    2    3    4 <NA>
494  498  480  497 1381

```

```
# frequency table of Q
```

Remove the helper variable.

```

sbs399 <- sbs399 |>
  select(-hrinc_rank)
# hrinc_rank was only needed to construct Q

```

Note that `ntile()` does the same in one step.

```
table(ntile(sbs399$hrinc, 4), useNA = "ifany")
```

```

  1    2    3    4 <NA>
493  492  492  492 1381

```

```
# dplyr::ntile() is a shortcut for the same quartile split
```

4.3.d Centering and standardizing variables

Create a variable `c_hrinc` that contains the centered hourly wage rate.

```

sbs399 <- sbs399 |>
  mutate(
    c_hrinc = hrinc - mean(hrinc, na.rm = TRUE)
  )
# center hrinc by subtracting the mean

```

Check that the mean of `c_hrinc` is approximately 0 and that the standard deviations of `c_hrinc` and `hrinc` are the same.

```
sbs399 |>
  summarise(
    mean_c_hrinc = mean(c_hrinc, na.rm = TRUE),
    sd_c_hrinc = sd(c_hrinc, na.rm = TRUE),
    sd_hrinc = sd(hrinc, na.rm = TRUE)
  )
```

```
# A tibble: 1 × 3
  mean_c_hrinc sd_c_hrinc sd_hrinc
      <dbl>      <dbl>    <dbl>
1   -1.13e-15      9.45      9.45
```

```
# mean of c_hrinc should be approximately 0; standard deviations should be
the same
```

Create a variable `z_hrinc` that contains the standardized hourly wage rate.

```
sbs399 <- sbs399 |>
  mutate(
    z_hrinc = (hrinc - mean(hrinc, na.rm = TRUE)) /
      sd(hrinc, na.rm = TRUE)
  )
# standardize hrinc
```

Check that the mean of `z_hrinc` is approximately 0 and the standard deviation is approximately 1.

```
sbs399 |>
  summarise(
    mean_z_hrinc = mean(z_hrinc, na.rm = TRUE),
    sd_z_hrinc = sd(z_hrinc, na.rm = TRUE)
  )
```

```
# A tibble: 1 × 2
  mean_z_hrinc sd_z_hrinc
      <dbl>      <dbl>
1   -1.17e-16      1
```

```
# mean should be approximately 0; standard deviation should be approximately 1
```

Check that `sex` has no missing values before grouping.

```
table(sbs399$sex, useNA = "ifany")
```

```
  0    1  
1699 1651
```

```
# if sex were missing for some respondents, those cases would form their own group  
# and z2_hrinc would be missing for them even though hrinc is observed
```

Create a variable `z2_hrinc` that standardizes hourly income separately for men and women.

```
sbs399 <- sbs399 |>  
  group_by(sex) |>  
  mutate(  
    z2_hrinc = (hrinc - mean(hrinc, na.rm = TRUE)) /  
               sd(hrinc, na.rm = TRUE)  
  ) |>  
  ungroup()  
# standardize hrinc separately by sex  
# this yields one variable, and it is missing only where hrinc is missing
```

Check the means and standard deviations by sex.

```
sbs399 |>  
  group_by(sex) |>  
  summarise(  
    mean_z2_hrinc = mean(z2_hrinc, na.rm = TRUE),  
    sd_z2_hrinc = sd(z2_hrinc, na.rm = TRUE)  
  )
```

```
# A tibble: 2 × 3  
  sex      mean_z2_hrinc sd_z2_hrinc  
<dbl>+<dbl>      <dbl>      <dbl>  
1 0 [man]      1.62e-16          1  
2 1 [woman]    1.76e-16          1
```

```
# within each sex, mean should be approximately 0 and standard deviation  
approximately 1
```

Apply a unit-range transformation to `hrinc`.


```
sbs399 <- sbs399 |>
  mutate(
    u_hrinc = (hrinc - min(hrinc, na.rm = TRUE)) /
      (max(hrinc, na.rm = TRUE) - min(hrinc, na.rm = TRUE))
  )
# unit-range transformation of hrinc
```

Verify that the transformed minimum equals 0 and the transformed maximum equals 1.

```
sbs399 |>
  summarise(
    min_u_hrinc = min(u_hrinc, na.rm = TRUE),
    max_u_hrinc = max(u_hrinc, na.rm = TRUE)
  )
```

```
# A tibble: 1 × 2
  min_u_hrinc max_u_hrinc
    <dbl>         <dbl>
1         0             1
```

```
# minimum should be 0 and maximum should be 1
```

Which method do you prefer for income? And for number of hours worked per week?

```
# For income, standardization is often useful because income can be very skewed.
# For hours worked per week, unit-range transformation can be easier to interpret.
# The best choice depends on the goal of the analysis.
```

4.3.e Creating additive scales

Explain why it is not sensible to simply add the five items.

```
# It is not sensible to simply add the five items if one item is coded in the
  opposite direction.
# In that case, high values do not mean the same thing for all items.
# The reversed item should first be recoded.
```

Which item should be reverse coded?

```
sbs399 |>
  select(sexroll1:sexrol5) |>
  cor(use = "complete.obs")
```

	sexrol1	sexrol2	sexrol3	sexrol4	sexrol5
sexrol1	1.0000000	0.4780910	0.2436255	0.3365474	-0.1620761
sexrol2	0.4780910	1.0000000	0.3093891	0.4325569	-0.2325207
sexrol3	0.2436255	0.3093891	1.0000000	0.3735788	-0.2195968
sexrol4	0.3365474	0.4325569	0.3735788	1.0000000	-0.2062143
sexrol5	-0.1620761	-0.2325207	-0.2195968	-0.2062143	1.0000000

```
# the reversed item is the one correlating negatively with the other four
# here this is sexrol5
```

Reverse item 5 using `case_when()`.

```
sbs399 <- sbs399 |>
  mutate(
    sexrol5r_case = case_when(
      sexrol5 == 1 ~ 5,
      sexrol5 == 2 ~ 4,
      sexrol5 == 3 ~ 3,
      sexrol5 == 4 ~ 2,
      sexrol5 == 5 ~ 1,
      TRUE ~ NA_real_
    )
  )
# reverse code sexrol5 using case_when()
```

Reverse item 5 using a mathematical expression.

```
sbs399 <- sbs399 |>
  mutate(
    sexrol5r_math = 6 - sexrol5
  )
# reverse code sexrol5 using a mathematical expression
```

Verify that both constructed variables are identical.

```
table(sbs399$sexrol5r_case, sbs399$sexrol5r_math, useNA = "ifany")
```

	1	2	3	4	5
1 2335	0	0	0	0	0
2 0 550	0	0	0	0	0
3 0 0 281	0	0	0	0	0

4	0	0	0	90	0
5	0	0	0	0	94

```
# the two reverse-coded variables should be identical
```

Create the attitude measure from the five items.

```
sbs399 <- sbs399 |>
  mutate(
    sexrole_scale = rowMeans(
      cbind(
        sexrol1,
        sexrol2,
        sexrol3,
        sexrol4,
        sexrol5r_math
      )
    )
  )
# create additive scale as row mean of the five items
# no na.rm is needed here because this dataset is complete
```

What is the mean of the measure?

```
sbs399 |>
  summarise(
    mean_sexrole_scale = mean(sexrole_scale, na.rm = TRUE)
  )
```

```
# A tibble: 1 × 1
  mean_sexrole_scale
              <dbl>
1                1.97
```

```
# mean of the sex role attitude scale
```

Can you interpret this mean substantively?

```
# The scale mean gives the average position on the sex role attitude items.
# A substantive interpretation depends on the direction of the item coding.
```

Open the dataset with missing values.

```
sbs399_sexroles <- read_dta("data/sbs399_sexroles.dta")
# open the dataset with missing values on the sex role items
```

Create the attitude measure using all available item responses.

```
sbs399_sexroles <- sbs399_sexroles |>
  mutate(
    sexrol5r_math = 6 - sexrol5,
    n_valid = rowSums(
      !is.na(
        cbind(
          sexrol1,
          sexrol2,
          sexrol3,
          sexrol4,
          sexrol5r_math
        )
      )
    ),
    sexrole_scale = if_else(
      n_valid == 0,
      NA_real_,
      rowMeans(
        cbind(
          sexrol1,
          sexrol2,
          sexrol3,
          sexrol4,
          sexrol5r_math
        )
      ),
      na.rm = TRUE
    )
  )
# create scale using the mean of available item responses
# if all items are missing, the scale is missing
```

Check the scale.

```
sbs399_sexroles |>
  summarise(
    mean_sexrole_scale = mean(sexrole_scale, na.rm = TRUE),
    min_sexrole_scale = min(sexrole_scale, na.rm = TRUE),
```

```
max_sexrole_scale = max(sexrole_scale, na.rm = TRUE)
)
```

```
# A tibble: 1 × 3
  mean_sexrole_scale min_sexrole_scale max_sexrole_scale
      <dbl>           <dbl>           <dbl>
1         1.97         1             5
```

```
# summary statistics for the scale with missing values handled using
rowMeans(na.rm = TRUE)
```

5 Selecting cases and repeating analyses for groups

In this part you will learn about the following basic tasks:

- How to select cases (observations), either permanently or temporarily, for one or more commands.
- How to repeat a command (or collection of commands) for various subgroups.

Function	Description
<code>filter()</code>	Select observations satisfying a condition
<code>slice()</code>	Select observations by row number
<code>slice_head()</code>	Select the first observations
<code>slice_tail()</code>	Select the last observations
<code>group_by()</code>	Define groups for subsequent operations
<code>.by</code>	Apply an operation separately within groups

5.1 R

Examples:

Select observations using logical conditions:

```
data |>
  filter(sex == 1)
# select observations for which sex equals 1

data |>
  filter(sex == 1, edu > 3)
# select observations for which sex equals 1 and edu is larger than 3
```

```
data |>
  filter(sex == 1 | edu >= 3)
# select observations for which sex equals 1 or edu is at least 3
```

Select observations based on row number:

Use `slice()` to select observations by their row number.

```
data |>
  slice(1:20)
# select the first 20 observations

data |>
  slice((n() - 19):n())
# select the last 20 observations
```

The same can also be done with `filter()` and `row_number()`:

```
data |>
  filter(row_number() <= 20)
# select the first 20 observations

data |>
  filter(row_number() >= n() - 19)
# select the last 20 observations
```

Keep a subset of observations:

```
data_male <- data |>
  filter(sex == 2)
# create a new dataset containing only observations for which sex equals 2

data_first100 <- data |>
  slice(1:100)
# create a new dataset containing only the first 100 observations
```

Remove observations:

```
data <- data |>
  filter(sex != 1)
# remove observations for which sex equals 1

data <- data |>
```

```
slice(-(1:100))  
# remove the first 100 observations
```

Compute statistics within groups:

```
data |>  
  summarise(  
    mean_income = mean(income, na.rm = TRUE),  
    .by = country ) # mean income by country
```

Compute statistics within combinations of groups:

```
data |>  
  summarise(  
    mean_income = mean(income, na.rm = TRUE),  
    .by = c(country, sex) ) # mean income by country and sex
```

Create variables within groups:

```
data <- data |>  
  mutate(  
    hinc = max(income, na.rm = TRUE),  
    .by = hrespnr ) # highest income within household
```

The `.by` argument is often a convenient alternative to `group_by()`. It applies the operation separately within groups without permanently changing the grouping structure of the dataset.

5.2 Assignments

In these assignments, you should use the dataset `sbs399`. Solutions can be found in `dh_select.Rmd`.

1. Using `filter()` and `nrow()`, count the number of observations in `sbs399`:
 - with age at most 30;
 - with age at least 20;
 - with age between 20 and 30, boundaries included;
 - with age between 20 and 30, boundaries excluded;
 - males over 25;
 - people who are either below 68 or female, with `sex` equal to 1.
2. Select the subjects with age at most 30 and store this subset in a new object. Do not overwrite the original dataset `sbs399`.

Count the number of subjects of age at least 20 in this subset.

Separately for men and women, count the number of subjects of age at least 20. Use `.by` or `group_by()` for this last question.

3. Reload the full dataset `sbs399`.

Create a selection variable, say `touse`, that takes the value 1 for males over 30 and 0 otherwise.

For the selected sample:

- create a two-way table;
- compute the correlation between the respondent's education, `educ`, and the respondent's father's education, `feduc`.

4. What is the difference between the following R commands?

```
data |>
  select(-sex)

data |>
  filter(sex != 1)

data |>
  filter(sex == 1)
```

5. Using the household identifier `hrespnr` in the dataset `sbs399`, use `.by` or `group_by()` to:

- define the number of participants in a household;
- list the oldest people in households without creating a new variable;
- list the ages of the same-sex households;
- list `sex` and `age` in increasing order of age in households where the age difference is larger than 5, without creating new variables;
- drop the singles.

5.3 Answers

Make sure this file is in the same R Project as the `data` folder. Use relative file paths, not full paths to your computer.

```
library(tidyverse)
library(haven)
library(janitor)
library(skimr)
```



```
# The file sbs399.dta should be available in your working directory.
sbs399 <- read_dta("data/sbs399.dta")
```

Check the coding of `sex` before using it in any condition.

```
table(sbs399$sex, useNA = "ifany")
```

```
  0    1
1699 1651
```

```
# sex = 0 for men and sex = 1 for women
```

The dataset comes from Stata, so the variable carries value labels. `as_factor()` makes them visible.

```
table(as_factor(sbs399$sex), useNA = "ifany")
```

```
  man woman
1699  1651
```

```
# the same table, now showing the labels instead of the codes
```

5.3.a 1. Counting observations with `filter()` and `nrow()`

Age at most 30.

```
sbs399 |>
  filter(age <= 30) |>
  nrow()
```

```
[1] 1090
```

```
# number of respondents aged 30 or younger
```

Age at least 20.

```
sbs399 |>
  filter(age >= 20) |>
  nrow()
```

```
[1] 3345
```

```
# number of respondents aged 20 or older
```

Age between 20 and 30, boundaries included.

```
sbs399 |>
  filter(age >= 20, age <= 30) |>
  nrow()
```

```
[1] 1085
```

```
# a comma between conditions means AND
# between(age, 20, 30) would give the same result
```

Age between 20 and 30, boundaries excluded.

```
sbs399 |>
  filter(age > 20, age < 30) |>
  nrow()
```

```
[1] 731
```

```
# respondents of exactly 20 or exactly 30 are now excluded
```

Males over 25.

```
sbs399 |>
  filter(sex == 0, age > 25) |>
  nrow()
```

```
[1] 1653
```

```
# men older than 25
```

People who are either below 68 or female.

```
sbs399 |>
  filter(age < 68 | sex == 1) |>
  nrow()
```

```
[1] 3330
```

```
# the vertical bar means OR, so respondents satisfying either condition are counted
```

Note that `filter()` drops rows for which the condition evaluates to `NA`. In this dataset `age` and `sex` are complete, so nothing is lost here, but with incomplete data the counts above would exclude those respondents.

5.3.b 2. Working with a subset

Select the subjects with age at most 30 and store them in a new object.

```
sbs399_young <- sbs399 |>
  filter(age <= 30)
# the original dataset sbs399 is left untouched
```

Count the number of subjects of age at least 20 in this subset.

```
sbs399_young |>
  filter(age >= 20) |>
  nrow()
```

```
[1] 1085
```

```
# respondents aged between 20 and 30 in the subset
```

The same count separately for men and women.

```
sbs399_young |>
  filter(age >= 20) |>
  summarise(
    n = n(),
    .by = sex
  )
```

```
# A tibble: 2 × 2
  sex      n
<dbl+lbl> <int>
1 0 [man]    476
2 1 [woman]  609
```

```
# counts by sex using .by
```

The same result with `group_by()`.

```
sbs399_young |>
  filter(age >= 20) |>
  group_by(sex) |>
  summarise(n = n())
```

```
# A tibble: 2 × 2
  sex      n
<dbl+lbl> <int>
1 0 [man]    476
2 1 [woman]  609
```

```
# group_by() followed by summarise() gives the same table
```

5.3.c 3. A selection variable

Reload the full dataset.

```
sbs399 <- read_dta("data/sbs399.dta")
# start again from the complete dataset
```

Create the selection variable `touse`.

```
sbs399 <- sbs399 |>
  mutate(
    touse = if_else(sex == 0 & age > 30, 1, 0)
  )
# touse = 1 for men older than 30, 0 otherwise
```

Check the selection variable.

```
table(sbs399$touse, useNA = "ifany")
```

```

      0      1
2130 1220

```

```
# how many observations are selected
```

Create the two-way table of education and father's education for the selected sample.

```

sbs399_selected <- sbs399 |>
  filter(touse == 1)

table(sbs399_selected$educ, sbs399_selected$feduc, useNA = "ifany")

```

	1	2	3	4	5	6	7	8
1	83	26	5	0	1	2	1	0
2	116	79	17	2	1	11	4	1
3	68	47	29	3	10	6	1	1
4	19	16	16	3	9	3	3	0
5	6	12	10	0	5	3	3	3
6	65	90	37	2	11	36	19	5
7	50	65	43	3	12	28	25	7
8	9	16	15	3	9	10	15	20

```
# cross-tabulation of educ and feduc within the selected sample
```

The same table with `janitor::tabyl()`, which adds the variable names and totals.

```

sbs399_selected |>
  tabyl(educ, feduc) |>
  adorn_totals(c("row", "col"))

```

educ	1	2	3	4	5	6	7	8	Total
1	83	26	5	0	1	2	1	0	118
2	116	79	17	2	1	11	4	1	231
3	68	47	29	3	10	6	1	1	165
4	19	16	16	3	9	3	3	0	69
5	6	12	10	0	5	3	3	3	42
6	65	90	37	2	11	36	19	5	265
7	50	65	43	3	12	28	25	7	233

	8	9	16	15	3	9	10	15	20	97
Total	416	351	172	16	58	99	71	37	1220	

```
# a more readable version of the same cross-tabulation
```

Compute the correlation between `educ` and `feduc` in the selected sample.

```
sbs399_selected |>
  summarise(
    r = cor(educ, feduc, use = "complete.obs")
  )
```

```
# A tibble: 1 × 1
      r
<dbl>
1 0.403
```

```
# use = "complete.obs" drops respondents with a missing value on either variable
# both variables are complete here, so this argument makes no difference
```

5.3.d 4. Comparing three commands

```
# select(-sex) removes the COLUMN sex; all rows are kept.
# filter(sex != 1) removes ROWS for which sex equals 1; the column stays.
# filter(sex == 1) keeps only the rows for which sex equals 1.
#
# The two filter() commands look like opposites, but they are not exact
# complements in general: rows with a missing value on sex are dropped by
# both, because NA != 1 and NA == 1 both evaluate to NA rather than TRUE.
```

Demonstrate this with the number of rows and columns.

```
dim(sbs399)
```

```
[1] 3350  51
```

```
dim(select(sbs399, -sex))
```

```
[1] 3350  50
```

```
dim(filter(sbs399, sex != 1))
```

```
[1] 1699  51
```

```
dim(filter(sbs399, sex == 1))
```

```
[1] 1651  51
```

```
# the first command changes the number of columns, the other two the number  
of rows
```

5.3.e 5. Working within households

Define the number of participants in a household.

```
sbs399 <- sbs399 |>  
  mutate(  
    n_household = n(),  
    .by = hrespnr  
  )  
# number of respondents sharing the same household identifier
```

Check the household sizes.

```
table(sbs399$n_household, useNA = "ifany")
```

```
  1    2  
288 3062
```

```
# frequency table of household sizes
```

List the oldest people in households, without creating a new variable.

```
sbs399 |>  
  filter(age == max(age), .by = hrespnr) |>  
  select(hrespnr, sex, age)
```

```
# A tibble: 1,985 × 3
  hrespnr sex      age
  <dbl> <dbl+lbl> <dbl>
1  11001 0 [man]    30
2  11002 0 [man]    33
3  11003 0 [man]    35
4  11004 0 [man]    34
5  11005 1 [woman]   35
6  11006 0 [man]    30
7  11007 0 [man]    36
8  11008 1 [woman]   34
9  11010 0 [man]    26
10 11011 1 [woman]   26
# i 1,975 more rows
```

```
# filter() with .by compares each respondent to the maximum age in their own
household
# households with a tie return more than one row
```

List the ages of the same-sex households.

```
sbs399 |>
  filter(n_distinct(sex) == 1, .by = hrespnr) |>
  select(hrespnr, sex, age)
```

```
# A tibble: 308 × 3
  hrespnr sex      age
  <dbl> <dbl+lbl> <dbl>
1  11001 0 [man]    30
2  11006 0 [man]    30
3  11010 0 [man]    26
4  11011 1 [woman]   26
5  11012 0 [man]    26
6  11013 1 [woman]   26
7  11014 0 [man]    26
8  11022 0 [man]    26
9  11024 0 [man]    26
10 11032 0 [man]    26
# i 298 more rows
```



```
# keep only households in which every member has the same value on sex
# note that single-person households satisfy this condition as well
```

List `sex` and `age` in increasing order of age, in households where the age difference is larger than 5.

```
sbs399 |>
  filter(max(age) - min(age) > 5, .by = hrespnr) |>
  arrange(hrespnr, age) |>
  select(hrespnr, sex, age)
```

```
# A tibble: 474 × 3
  hrespnr sex      age
  <dbl> <dbl+lbl> <dbl>
1  11002 1 [woman]    26
2  11002 0 [man]     33
3  11026 1 [woman]    23
4  11026 0 [man]     34
5  11051 1 [woman]    28
6  11051 0 [man]     34
7  11078 0 [man]     26
8  11078 1 [woman]    35
9  11122 1 [woman]    34
10 11122 0 [man]     44
# i 464 more rows
```

```
# the range of age is computed within each household without storing it
```

Drop the singles.

```
sbs399 <- sbs399 |>
  filter(n() > 1, .by = hrespnr)
# keep only respondents living in a household with at least two participants
```

Check the result.

```
table(sbs399$n_household, useNA = "ifany")
```

```
  2
3062
```

```
# households of size 1 should no longer appear
```